第 1-2 讲 • 一切都是文件

> **模块 1 • 程序到底是什么** ｜ 建议用时 **40 分钟**（阅读 22 + 动手 18）
> 前置要求：完成 1-1（理解进程与内存）

本节目标

读完并做完这一讲，你能：

1. 说清**文件、文件夹、路径**三者的关系，并区分**绝对路径**和**相对路径**。
2. 理解**文件扩展名**的作用：它不决定内容，只决定"谁来打开它"。
3. 讲清**编码**是什么，并解释一个你电脑上**真实存在**的问题：为什么配置里写着 `D:\鍔犻€熷櫒` 而不是 `D:\加速器`。
4. 知道"文本文件"和"二进制文件"的区别，以及为什么数据库文件不能用记事本打开。
5. 能自己动手查看和解释记账簿的配置文件。

一、文件、文件夹、路径

1.1 现实比喻：档案柜

- **文件** = 一张纸，上面写着内容。
- **文件夹** = 一个抽屉，可以装纸，也可以装别的抽屉。
- **路径** = 从柜子最外层走到这张纸的**完整路线**。

```
C:\Users\34123\Desktop\记账簿\package.json
└┬┘└──────┬──────┘└──┬──┘└──┬──┘└──────┬──────┘
 │        │          │      │          └ 文件本身
 │        │          │      └ 文件夹（项目的家）
 │        │          └ 文件夹
 │        └ 用户目录
 └ 盘符：从哪个盘开始
```

反斜杠 `\` 是 Windows 的分隔符（Linux/macOS 用 `/`）。这个差异以后会让你踩坑——写路径时要注意。

1.2 绝对路径 vs 相对路径

类型	长相	特点
绝对路径	`C:\Users\34123\Desktop\记账簿\package.json`	从盘符开始，**独一无二**，换到哪个文件夹都指向同一个文件
相对路径	`src\main\index.ts`	相对于"你现在站的位置"，**换个地方意思就变了**

这个区别极其重要，因为它是很多"找不到文件"报错的根源。

项目里的真实例子：`package.json` 里写着

```
"main": "./out/main/index.js"
```

那个 `./` 就是"相对路径"的标志，意思是"**从项目根目录开始算**"。所以不管你把这个项目放在桌面还是 D 盘，它都能找到自己的入口文件。**如果这里写成 `C:\Users\34123\...` 这样的绝对路径，项目一搬到别的电脑就废了。**

> **记住这条经验**：项目内部的路径一律用相对路径，只有"用户的数据放在哪"这种才用绝对路径。

1.3 `..` 和 `.`

- `..` = **上一层**（上一讲你已经用过 `cd ..`）
- `.` = **当前这一层**

所以 `.\src\main` 和 `src\main` 是一个意思。

二、扩展名：它只是"标签"，不是"内容"

2.1 扩展名决定"谁来打开"

`report.txt` 里的 `.txt` 就是扩展名。它的作用只有一个：**告诉系统该用哪个程序打开它。**

扩展名	通常是谁	在你项目里的数量
`.ts`	TypeScript 源码（主进程/共享层）	74 个
`.tsx`	TypeScript + 界面（渲染进程）	26 个
`.css`	样式	2 个
`.json`	配置清单	若干
`.db`	SQLite 数据库	1 个（你的账本）
`.exe`	可执行程序	安装包里

关键认知：扩展名不改变内容，只影响"双击时谁来打开"。 你可以把一个 `.txt` 改名成 `.jpg`，内容一个字没变，只是系统会试着用看图软件打开它，然后失败。

2.2 文本文件 vs 二进制文件

这是本讲第二个重要概念：

	文本文件	二进制文件
内容	人能看懂的字符（`{ "name": "记账簿" }`）	机器编码的字节
例子	`.ts` `.json` `.md` `.txt` `.csv`	`.db` `.exe` `.png` `.onnx`
能用记事本打开吗	能	**不能（打开是乱码）**

你的账本 `daily-ledger.db` 是二进制文件。 千万别用记事本打开它再保存——那会把数据库彻底弄坏。想查看里面的内容，要用专门的工具（第 2-3 讲会给你工具）。

怎么一眼认出它是二进制：用下面这段命令看它的开头几个字节（动手任务里会做）：

```
Format-Hex -Path "D:\加速器\daily-ledger.db" -Count 16
```

你会看到开头是 `53 51 4C 69 74 65 20 66 6F 72 6D 61 74 20 33` —— 翻译成文字是 `SQLite format 3`。这是数据库文件的"身份证"，前 16 个字节固定这样，用来告诉程序"我是一个 SQLite 数据库"。对比一下：文本文件的开头会直接是 `{` 或 `#` 这种能看懂的东西。

三、编码：为什么你的配置里写着乱码

3.1 先看一个真实的现象

打开这个文件：

```
C:\Users\34123\AppData\daily-ledger\config.json
```

（注意：`AppData` 是隐藏文件夹，要在文件资源管理器里勾选「显示隐藏的项目」才能看到。）

你会看到：

```
{
  "initialized": true,
  "dataDir": "D:\\鍔犻€熷櫒"
}
```

"鍔犻€熷櫒"是什么？ 是乱码吗？—— **是的，但它是"显示层面"的乱码，实际数据完好无损。**

我实测验证过这一点（你可以在动手环节自己复现）：

内容	值
你数据目录的真实名字	`加速器`
这三个字的 **UTF-8** 字节	`E5 8A A0 E9 80 9F E5 99 A8`
把上面这串字节**按 GBK 解码**	`鍔犻€熷櫒` ← 和 config.json 里显示的**一模一样**
用 UTF-8 正确读取同一个文件	`"dataDir": "D:\\加速器"` ✅ 完全正常

结论：文件里存的字节是对的，只是**某个程序用错的规则去读了它**。你的记账数据一个字都没丢。

3.2 为什么会这样：编码是"翻译规则"

计算机只认**数字**。文字要存进文件，必须有一套"文字 ↔ 数字"的对照规则，这套规则就叫**编码（Encoding）**。

- 中文有 6 万多个字，一字节（8 位，最多 256 种）装不下，所以要用**多个字节**表示一个字。
- 而"多个字节怎么拼回一个字"，有**好几套不同的规则**：UTF-8、GBK、GB18030、Big5……

关键问题：同一串字节，用不同规则去读，会读出完全不同的字。

举个例子：`加速器` 这三个字用 UTF-8 编码后是一串字节 `E5 8A A0 E9 80 9F E5 99 A8`， 如果**有人用 GBK 规则去读这串字节**，就会读出 `鍔犻€熷櫒`。

所以：

说法	对不对
"你的数据坏了"	❌ 数据完好
"写入时用了 UTF-8，读取时用了 GBK"	✅ 这才是真相
"换个用 UTF-8 的程序打开就正常"	✅ 对

3.3 这对你意味着什么

好消息：这个乱码**不影响任何功能。**记账簿打开数据库用的是真实路径（程序内部读的是对的），只是那个 JSON 文件在**某些程序**里显示成乱码。

为什么会出现：这是**当时写入这个文件的程序用了某种编码，而你现在用来查看的工具用了另一种。**本质上是个"翻译不一致"问题。

你要记住的实用规则：

1. **给程序读的文件（配置、代码），统一用 UTF-8。** 这是当今的通用标准。
2. **给人看的文件如果出现乱码，先怀疑编码，不要怀疑数据损坏。**
3. **看到"乱码"别慌**——先问"是谁用什么编码读的"，通常换一个工具就好了。

> 这个坑我自己也踩过：做讲义 PDF 时，用代码加载一个带中文名的文件，报错 `ERR_FAILED`。
> 排查半天，最后发现不是编码问题，而是**加载方式**的问题。我在讲义里讲这一课，正是因为**这类问题在你以后开发中会反复出现**。
> 排查时的正确姿势是：**先看真实报错，再一个个排除假设**，而不是凭直觉猜（第 4-4 讲专门讲这个）。

四、在项目里看真实文件

4.1 记账簿的两个关键目录

① 程序目录（软件本体，你平时不用管）

```
C:\Users\34123\AppData\Local\Programs\daily-ledger\
├─ 记账簿.exe            ← 真正的程序
└─ resources\            ← 打包进去的资源（图标、OCR 模型）
```

② 数据目录（你的账，最重要）

```
D:\加速器\
├─ daily-ledger.db       ← 全部账目（二进制！不要用记事本开）
├─ backup\               ← 自动备份
└─ LieBao\               ← 收据图片
```

为什么分开：这样卸载软件**不会删掉你的账。**这是本地软件的重要设计原则——**程序和数据分离。**

4.2 引导文件：程序怎么知道数据在哪

程序启动时要先知道"数据在哪个目录"，这个信息存在：

```
C:\Users\34123\AppData\daily-ledger\config.json
```

内容就是刚才那个 `dataDir` 字段。

为什么不把数据目录硬写在程序里：因为你想换地方存（比如换到 D 盘、或放到网盘同步目录）。用配置文件，就能随时改而不用改程序。

这正是"数据与程序分离"的落地方式。

五、动手任务

步骤 1：看见隐藏的 AppData

1. 打开文件资源管理器，按 `Win + R`，输入 `%APPDATA%`，回车。
2. 你会进到 `C:\Users\34123\AppData\Roaming`。
3. 找 `daily-ledger` 文件夹，进去看 `config.json`。

注意：`%APPDATA%` 这种写法叫**环境变量**——它是"某个固定位置的代号"。这样写的好处是不用记完整路径。

步骤 2：用终端"读懂"这个文件

在 PowerShell 里执行（把它当成"用 UTF-8 正确读取"的示范）：

```
cd "C:\Users\34123\Desktop\记账簿"
Get-Content "$env:APPDATA\daily-ledger\config.json" -Encoding UTF8
```

观察：这次的显示效果和你在记事本里看到的一样吗？试着把 `-Encoding UTF8` 去掉，再执行一次，对比结果。

这个对比就是本讲的核心：同一份文件，换个读取方式，显示就不一样。

步骤 3：验证"扩展名只是标签"

在桌面上新建一个文本文件 `test.txt`，里面写一句话。然后：

```
cd "$env:USERPROFILE\Desktop"
Rename-Item test.txt test.jpg
```

观察：文件图标变了（系统以为它是图片），双击它——打开失败或用看图软件打开显示"无法打开"。

结论：内容没变，只是"谁来打开"变了。**这个实验让你彻底记住：扩展名是标签，不是内容。**

（做完删掉它。）

步骤 4：找到入口文件，验证相对路径

```
cd "C:\Users\34123\Desktop\记账簿"
Get-Content package.json | Select-String "main"
```

预期看到：`"main": "./out/main/index.js"`。

思考：为什么这里写着 `./`（相对路径）而不是完整路径？

答案：因为这个项目要能在任何人的电脑上运行。写死 `C:\Users\34123\...` 的话，别人拿到就找不到文件了。**这是"可移植性"的最基本要求。**

步骤 5：确认数据还在（安心检查）

```
Get-ChildItem "D:\加速器"
```

预期看到：`daily-ledger.db`、`backup`、`LieBao` 等。

再执行：

```
(Get-Item "D:\加速器\daily-ledger.db").Length
```

这会打印出数据库文件的字节数（一个数字）。只要它不是 0，你的账就在。

这一课的意义：以后你改代码、做实验之前，都可以先跑一下这个命令**确认数据文件的字节数**，作为"我动手前它是好的"的凭据。这是养成"操作前留证据"习惯的第一步。

步骤 6：亲眼看见"文本文件"和"二进制文件"的差别

先看数据库的开头：

```
Format-Hex -Path "D:\加速器\daily-ledger.db" -Count 16
```

预期看到：`53 51 4C 69 74 65 20 66 6F 72 6D 61 74 20 33 00`，右侧注释会显示 `SQLite format 3`。

再看一个文本文件的开头：

```
Format-Hex -Path "C:\Users\34123\Desktop\记账簿\package.json" -Count 16
```

预期看到：`7B 0D 0A ...`，右侧会直接显示出 `{` —— **能看懂的字符。**

对比结论：

文件	前 16 字节看起来	类型
`daily-ledger.db`	一串十六进制码，看不懂	**二进制**（有固定格式）
`package.json`	直接显示 `{ "name": ...`	**文本**（人能读）

这就是"二进制文件不能用记事本改"的直观证据：记事本只会把那些字节当文字翻译，然后你保存时它又按文字重新编码写回去——**格式就被破坏了。**

卡住了怎么办

现象	原因	怎么办
找不到 `AppData` 文件夹	它是隐藏文件夹	资源管理器 → 「查看」→ 勾选「隐藏的项目」
`config.json` 显示乱码	读取工具用了不同编码	正常现象，用 `-Encoding UTF8` 读；**不要因此以为数据坏了**
改了扩展名后改不回来	资源管理器隐藏了扩展名	先勾选「文件扩展名」，再重命名
`$env:APPDATA` 没反应	用错了引号类型	PowerShell 里用双引号 `"$env:APPDATA\..."`
`D:\加速器` 不存在	数据目录可能被改了	读 `config.json` 里的 `dataDir` 看真实位置

六、自测题

1. 绝对路径和相对路径有什么区别？为什么项目里要用相对路径？

2. 把 `photo.jpg` 改名成 `photo.txt`，图片内容会变吗？会发生什么？

3. 你的 `config.json` 里显示 `D:\鍔犻€熷櫒`，你的账坏了吗？为什么？

4. 为什么不能用记事本打开 `daily-ledger.db`？

5. 记账簿的程序目录和数据目录为什么分开？这样做有什么好处？

答案与解析

1. 绝对路径**从盘符开始**，在任何位置都指向同一个文件；相对路径**相对于当前位置**，换个地方含义就变。→ 项目用相对路径是为了**可移植**：换到任何人的电脑、放在任何目录都能正常找到文件。写死绝对路径的项目一搬家就废。

2. 内容完全不变，只是一行字节。变的只是"系统认为该用哪个程序打开它"。 → 这个实验证明：**扩展名是标签，不是内容。**

3. 没坏。 那是"写入时用一种编码、读取时用另一种"造成的**显示问题**，实际的字节序列没问题，程序按真实路径读数据库一切正常。 → 记住：**看到乱码先怀疑编码，不要怀疑数据损坏。**

4. 因为它是**二进制文件**，不是给人看的文本。记事本会把它当文字解析，显示成乱码；如果你**保存**了，记事本会按文本重新编码写回，**数据库就被破坏了。** → 要查看数据库内容要用专门工具（第 2-3 讲提供）。

5. 程序目录放软件本体（`AppData\Local\Programs`），**数据目录**放你的账（`D:\加速器`），两者的位置记录在 `config.json` 里。 → 好处：**卸载、重装、升级软件都不会碰你的数据**；你还可以自由把数据换到别的盘或网盘同步目录。这是"数据与程序分离"原则。

七、术语表

术语	一句话解释
文件 / File	硬盘上存内容的基本单位
文件夹 / 目录	装文件和其它文件夹的容器
路径 / Path	定位文件或文件夹的"路线"
绝对路径	从盘符写起的完整路径，唯一
相对路径	相对于当前位置的路径，随位置变化
`..` / `.`	上一层 / 当前层
扩展名	文件名最后的 `.xxx`，决定用哪个程序打开
文本文件	内容是人能看懂的字符
二进制文件	内容是按特定格式编码的字节，不能用记事本编辑
编码 / Encoding	文字与数字之间的对照规则（UTF-8、GBK 等）
乱码	用错误的编码规则去读字节造成的显示异常，**通常数据没坏**
UTF-8	当今通用编码标准，跨平台、跨语言都推荐用它
环境变量	某个固定位置的代号，如 `%APPDATA%`、`$env:USERPROFILE`
可移植性	项目能在不同电脑/目录正常运行的能力
数据与程序分离	软件本体和用户数据分开放，互不影响

八、本讲与项目的对应

- 你查看的文件：`%APPDATA%\daily-ledger\config.json`（引导文件）、`package.json`（入口路径）
- 你确认的数据：`D:\加速器\daily-ledger.db` 的字节数

- **本讲不修改任何项目代码，也不会影响你的记账数据**
- 上一讲：1-1 双击之后发生了什么 ｜ 下一讲：**1-3 代码是怎么变成软件的**（源码 → 打包 → exe）

讲义版本：v1 ｜ 对应项目版本：0.1.0